

Advanced Reading in Computer Vision (MAT3563)

LAB 05. ONE-STAGE OBJECT DETECTION MODELS

Trong bài thực hành này, chúng ta tiếp tục tìm hiểu về 1-stage object detection xoay quanh SSD và YOLO. Tương tự Lab trước, mục tiêu của bài học này là:

- (1) Hiểu pipeline 1-stage (YOLO/SSD): **FPN** → **Head phân loại & hồi quy bbox** → **NMS** (không dùng RoI Pool/Align như 2-stage; phần này chỉ nêu để so sánh).
- (2) Tự code các khối cốt lõi: IoU, encode/decode bbox (SSD), anchor/grid (YOLO/SSD), NMS/Soft-NMS, FPN, loss cơ bản.
- (3) Chạy inference với pretrained YOLOv8 & SSD300.
- (4) Fine-tune nhanh: (A) YOLOv8 (PyTorch); (B) SSD (torchvision); (C) SSD MobileNet V2 (TensorFlow OD API).

Sau bài thực hành này, học viên cần làm chủ được việc tinh chỉnh/huấn luyện lại mô hình SSD/các version YOLO trên một tập dữ liệu mới (có thể khác về tập nhãn, khác về miền nội dung ảnh – ví dụ ảnh y tế...).

1. MỤC TIÊU BÀI THỰC HÀNH & YÊU CẦU MÔI TRƯỜNG

Các bạn cần nắm được:

- Kiến trúc và luồng dữ liệu qua FPN → Anchor Based/Anchor Free → Heads → NMS/Soft-NMS cho SSD và YOLO.
- Thực hành từng “mắt xích” bằng các script nhỏ, nhìn thấy đầu-vào/đầu-ra.
- Chạy **pretrained inference** (với cả thư viện Torch & TF).
- **Fine-tune** trên dữ liệu Pascal VOC-style (XML) hoặc COCO-style (json).

Môi trường gợi ý:

- Python ≥ 3.9, CUDA nếu có.
- PyTorch ≥ 2.2, torchvision ≥ 0.17.
- TensorFlow ≥ 2.12 (GPU nếu có).
- (Tuỳ chọn, để fine-tune TF) TensorFlow Object Detection API.

2. DỮ LIỆU & CẤU TRÚC THƯ MỤC MẪU

Tương tự Lab04

3. MINI-LABS (HIỂU TỪNG MODULE CỐT LÕI QUA VÍ DỤ NHỎ)

Mỗi mini-lab là một file Python độc lập (chạy được trong notebook). **Dữ liệu minh họa**: dùng 1–2 ảnh bất kỳ (ảnh đường phố, bản đồ vệ tinh...) lưu ở đường dẫn để vẽ boxes. Thay đường dẫn của ví dụ trong bài thành đường dẫn đến ảnh của bạn. Mỗi lab độc lập, bạn có thể chạy riêng hoặc tổng hợp thành 01 file chạy các ví dụ minh họa cho các block được đề cập (tham khảo one_stage_ObjDetect_module_blocks.ipynb). Trước hết ta xây dựng các module cốt lõi, một số sẽ tương tự như của mô hình 2-stages đã học trong Lab04 và đã được trình bày trong bài giảng lý thuyết nên sẽ không có phần giải thích, học viên tự tìm hiểu ý nghĩa, vai trò từng module.

A. CODE RIÊNG RẼ CHO CÁC MODULE CỐT LÕI

A.1 IoU & GloU/DIoU/CIoU + Encode/Decode (SSD)

```
import torch

def box_xyxy_to_xywh(b):
    x1, y1, x2, y2 = b.unbind(-1)
    w = x2 - x1; h = y2 - y1
    cx = x1 + 0.5*w; cy = y1 + 0.5*h
    return torch.stack([cx, cy, w, h], dim=-1)

def box_xywh_to_xyxy(b):
```

```

cx,cy,w,h = b.unbind(-1)
x1 = cx - 0.5*w; y1 = cy - 0.5*h
x2 = cx + 0.5*w; y2 = cy + 0.5*h
return torch.stack([x1,y1,x2,y2], dim=-1)

def iou(boxes1, boxes2):
    # boxes*: [N,4] (xyxy)
    tl = torch.max(boxes1[:,None,:2], boxes2[:,,:2]) # [N,M,2]
    br = torch.min(boxes1[:,None,2:], boxes2[:,2:])
    wh = (br - tl).clamp(min=0)
    inter = wh[...,0]*wh[...,1]
    area1 = (boxes1[:,2]-boxes1[:,0])*(boxes1[:,3]-boxes1[:,1])
    area2 = (boxes2[:,2]-boxes2[:,0])*(boxes2[:,3]-boxes2[:,1])
    union = area1[:,None] + area2 - inter
    return inter / (union + 1e-7)

# SSD encode/decode (theo tỉ lệ offsets và log-scale)
def ssd_encode(gt_xyxy, anchors_xywh, variances=(0.1,0.2)):
    gt_xywh = box_xyxy_to_xywh(gt_xyxy)
    g_cxcy = (gt_xywh[:,2] - anchors_xywh[:,2]) / anchors_xywh[:,2:]
    g_wh = torch.log(gt_xywh[:,2:] / anchors_xywh[:,2:])
    v1,v2 = variances
    return torch.cat([(g_cxcy/v1), (g_wh/v2)], dim=-1)

def ssd_decode(pred_t, anchors_xywh, variances=(0.1,0.2)):
    v1,v2 = variances
    cxcy = pred_t[:,2]*v1*anchors_xywh[:,2:] + anchors_xywh[:,2:]
    wh = torch.exp(pred_t[:,2:]*v2)*anchors_xywh[:,2:]
    return box_xywh_to_xyxy(torch.cat([cxcy,wh], dim=-1))

```

A.2 NMS/Soft-NMS

```

import torch

def nms(boxes, scores, iou_th=0.5):
    keep = []
    idxs = scores.argsort(descending=True)
    while idxs.numel() > 0:
        i = idxs[0]; keep.append(i.item())
        if idxs.numel() == 1: break
        iou_vals = iou(boxes[i].unsqueeze(0), boxes[idxs[1:]]).squeeze(0)
        idxs = idxs[1:][iou_vals <= iou_th]
    return keep

# Soft-NMS (Gaussian)
def soft_nms(boxes, scores, iou_th=0.5, sigma=0.5, score_th=0.001):
    idxs = scores.argsort(descending=True)
    boxes = boxes.clone(); scores = scores.clone()
    keep = []
    while idxs.numel() > 0:
        i = idxs[0]; keep.append(i.item())
        if idxs.numel() == 1: break
        others = idxs[1:]
        ov = iou(boxes[i].unsqueeze(0), boxes[others]).squeeze(0)
        weights = torch.exp(-(ov**2) / sigma)
        scores[others] = scores[others]*weights
        idxs = torch.cat([others[scores[others] > score_th]])
        idxs = idxs[scores[idxs].argsort(descending=True)]

    return keep

```

A.3 Anchor/Grid (SSD & YOLO)

```

import torch
def ssd_anchors(feat_h, feat_w, img_h, img_w, scales=[0.1,0.2,0.35], ratios=[1.0,2.0,0.5]):
    # Trả về anchors ở dạng (cx,cy,w,h) tỷ lệ theo pixel
    stride_y = img_h/feat_h; stride_x = img_w/feat_w
    anchors=[]
    for y in range(feat_h):
        for x in range(feat_w):
            cy = (y+0.5)*stride_y; cx=(x+0.5)*stride_x
            for s in scales:
                for r in ratios:
                    w = img_w*s*(r**0.5); h = img_h*s/(r**0.5)
                    anchors.append([cx,cy,w,h])
    return torch.tensor(anchors, dtype=torch.float32)

```

```
def yolo_grid(feats_h, feats_w, stride):
    # grid (cx, cy) tính theo pixel (YOLO head dự đoán offset trên grid)
    yv, xv = torch.meshgrid(torch.arange(feats_h), torch.arange(feats_w), indexing='ij')
    grid = torch.stack((xv, yv), 2).view(-1,2).float()

    return (grid + 0.5) * stride # tâm ô lưới
```

A.4 FPN (top-down + lateral) — khung tối giản

```
import torch.nn as nn

class SimpleFPN(nn.Module):
    def __init__(self, c3, c4, c5, out_channels=256):
        super().__init__()
        self.lateral3 = nn.Conv2d(c3, out_channels, 1)
        self.lateral4 = nn.Conv2d(c4, out_channels, 1)
        self.lateral5 = nn.Conv2d(c5, out_channels, 1)
        self.smooth3 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.smooth4 = nn.Conv2d(out_channels, out_channels, 3, padding=1)

    def forward(self, C3, C4, C5):
        P5 = self.lateral5(C5)
        P4 = self.lateral4(C4) + nn.functional.interpolate(P5, scale_factor=2, mode='nearest')
        P3 = self.lateral3(C3) + nn.functional.interpolate(P4, scale_factor=2, mode='nearest')
        P4 = self.smooth4(P4); P3 = self.smooth3(P3)

        return P3, P4, P5 # có thể tạo thêm P6,P7 bằng stride conv
```

A.5 Class-specific vs class-agnostic heads; Loss (YOLO/SSD)

- **SSD head:** conv 3×3 cho **cls** (k×num_anchors) và **bbox** (4×num_anchors) trên mỗi mức FPN.
- **YOLO head:** mỗi ô lưới/mức FPN dự đoán [tx,ty,tw,th,obj, p(class)...].
- **Loss:**
 - SSD: **Smooth-L1/L1** cho bbox + **Cross-Entropy** cho cls, hard negative mining.
 - YOLO: **CloU/GIoU/DIoU** cho bbox, **BCE** cho obj/cls, focal-like weighting tùy biến.

Skeleton SSD head (PyTorch):

```
class SSDHead(nn.Module):
    def __init__(self, in_channels, num_anchors, num_classes):
        super().__init__()
        self.cls = nn.Conv2d(in_channels, num_anchors*num_classes, 3, padding=1)
        self.reg = nn.Conv2d(in_channels, num_anchors*4, 3, padding=1)

    def forward(self, feats): # feats: list[P3,P4,P5...]
        cls_outs, reg_outs = [], []
        for f in feats:
            cls_outs.append(self.cls(f).permute(0,2,3,1).contiguous())
            reg_outs.append(self.reg(f).permute(0,2,3,1).contiguous())

        return cls_outs, reg_outs
```

Ghi chú về RoI Pool/Align: Đây là module tiêu biểu của 2-stage (R-CNN/Faster R-CNN), **không** dùng trong 1-stage. Ở đây chỉ nhắc để phân biệt. Với 1-stage, việc “lấy vùng” diễn ra gián tiếp thông qua **anchors/grid + Regression head**.

B. CODE THỰC NGHIỆM CÁC MODULE CỐT LÕI TRÊN ẢNH

Dưới đây là các **mini-lab thực nghiệm** chạy trực tiếp trên **01 ảnh đầu vào** (ví dụ input/street.jpg) cho từng mô-đun cốt lõi. Mỗi lab đều:

1. chuẩn bị ảnh & ground-truth (GT) box – có thể căn cứ theo các đối tượng trong ảnh để tạo GT box có nghĩa;
2. chạy Module;
3. trực quan hóa ngay kết quả.

Gợi ý: chèn các cell này vào **notebook** bạn đã tải. Các hàm tiện ích (IoU, NMS, SimpleFPN, ...) có thể được chèn lại vào phần chạy tổng hợp ở phía sau.

B.0 Chuẩn bị chung (ảnh + GT box + tool vẽ)

```
import os, cv2, numpy as np, torch
from PIL import Image, ImageDraw

root_dir = "D:/Teach_n_Train/Advanced_Lessons_CV/LABS/Object_Detection/Lab05/"
IMG_PATH = root_dir + 'input/street.jpg'

if not os.path.exists(IMG_PATH):
    img = Image.new('RGB', (960, 640), (235, 240, 245))
    d = ImageDraw.Draw(img)
    for (x,y,w,h) in [(120,300,120,80), (420,280,90,70), (700,320,160,100)]:
        d.rectangle([x,y,x+w,y+h], outline=(60,60,60), width=4)
        d.text((20,20), "Dummy Street-like Scene", fill=(10,10,10))
    img.save(IMG_PATH)

# GT boxes (xyxy) - ví dụ tay (bạn thay bằng GT thật nếu có)
GT = np.array([
    [90, 780, 210, 965], # object 1
    [500, 760, 615, 860], # object 2
    [1100, 750, 1230, 875], # object 3
], dtype=np.float32)

def draw_boxes(path, boxes, color=(0,255,0), labels=None, scores=None, out='runs_demo/out.jpg',
th=0.0):
    os.makedirs(root_dir + 'runs_demo', exist_ok=True)
    im = cv2.imread(path)
    for i,b in enumerate(boxes):
        if scores is not None and scores[i] < th:
            continue
        x1,y1,x2,y2 = map(int, b)
        cv2.rectangle(im, (x1,y1), (x2,y2), color, 2)
        if labels is not None or scores is not None:
            txt=''
            if labels is not None: txt += str(labels[i])
            if scores is not None: txt += f" {scores[i]:.2f}"
            if txt: cv2.putText(im, txt, (x1,max(0,y1-6)), cv2.FONT_HERSHEY_SIMPLEX,
                                0.6, color, 2)
    cv2.imwrite(out, im)

print('Saved:', out)
```

B.1 IoU & GIoU/DIoU/CIoU + Encode/Decode (SSD)

Mục tiêu

- Tính IoU giữa GT và *anchors* gần nhất.
- Thử **encode** GT → offsets theo anchor; rồi **decode** ngược lại để kiểm tra sai số.

```
# anchors demo (tạo grid đơn giản)
im = cv2.imread(IMG_PATH)
H, W, chal_ = im.shape

feat_h, feat_w = 8, 10 # stride ~32
stride_y, stride_x = H/feat_h, W/feat_w
scales = [0.15, 0.25]; ratios=[1.0, 2.0, 0.5]
anchors=[]

for y in range(feat_h):
    for x in range(feat_w):
        cy=(y+0.5)*stride_y; cx=(x+0.5)*stride_x
        for s in scales:
            for r in ratios:
                w=W*s*(r**0.5); h=H*s/(r**0.5)
                anchors.append([cx-0.5*w, cy-0.5*h, cx+0.5*w, cy+0.5*h])
anchors = torch.tensor(anchors, dtype=torch.float32)

gt = torch.tensor(GT, dtype=torch.float32)
IOU = iou(gt, anchors) # [n_gt, n_anchor]

print(IOU.shape)

best_idx = IOU.argmax(dim=1) # anchor tốt nhất cho mỗi GT
best_anchors = anchors[best_idx]
```

```
# SSD encode/decode
def ssd_encode(gt_xyxy, anc_xyxy, variances=(0.1,0.2)):
    v1,v2 = variances
    gt_xywh = box_xyxy_to_xywh(gt_xyxy); anc_xywh = box_xyxy_to_xywh(anc_xyxy)
    t_xy = (gt_xywh[:, :2]-anc_xywh[:, :2]) / anc_xywh[:, :2]
    t_wh = torch.log(gt_xywh[:, 2:]/anc_xywh[:, 2:])
    return torch.cat([t_xy/v1, t_wh/v2], dim=-1)

def ssd_decode(t, anc_xyxy, variances=(0.1,0.2)):
    v1,v2 = variances
    anc_xywh = box_xyxy_to_xywh(anc_xyxy)
    cxcy = t[:, :2]*v1*anc_xywh[:, 2:] + anc_xywh[:, :2]
    wh = torch.exp(t[:, 2:]*v2)*anc_xywh[:, 2:]
    return box_xywh_to_xyxy(torch.cat([cxcy, wh], dim=-1))

t = ssd_encode(gt, best_anchors)
gt_rec = ssd_decode(t, best_anchors)
print('Max L1 decode error:', (gt_rec-gt).abs().max().item())

# Vẽ GT (xanh) & anchor tốt nhất (vàng) & decode (lục)
draw_boxes(IMG_PATH, anchors.numpy(), color=(0,0,255), out=root_dir
+'runs_demo/a1_anchors.jpg')

draw_boxes(IMG_PATH, GT, color=(0,255,0), out=root_dir +'runs_demo/a1_gt.jpg')
draw_boxes(IMG_PATH, best_anchors.numpy(), color=(0,255,255), out=root_dir +
'runs_demo/a1_best_anchors.jpg')

draw_boxes(IMG_PATH, gt_rec.numpy(), color=(255,0,255), out=root_dir +
'runs_demo/a1_decoded.jpg')
```

Kỳ vọng: IoU hiển thị, sai số decode ~ rất nhỏ (vì encode/decode đúng).

Bài tập 0.1: thay feat_h/w, scales/ratios xem anchor “bắt” GT tốt hơn.

B.2 NMS vs Soft-NMS trên cùng ảnh (dùng boxes giả lập)

Mục tiêu: Cho thấy khả năng của **SOFT-NMS** giữ lại các bounding box có thể chứa đối tượng khác nhau, cho dù cùng class, có confidence thấp hơn và các boundingbox trùng nhau rất nhiều (IoU cao). **NMS** gốc sẽ loại bỏ Bbox có confidence thấp hơn trong trường hợp này.

Kỳ vọng: Soft-NMS giữ thêm box hợp lý hơn NMS khi chồng lấn cao.

```
# boxes “raw” bị chồng lấn + confidence
boxes = torch.tensor([
    [93, 781, 212, 960], # gần GT1
    [1080, 755, 1200, 869], # trùng mạnh
    [503, 758, 611, 862], # gần GT2
    [1097, 752, 1228, 873], # gần GT3
], dtype=torch.float32)

scores = torch.tensor([0.88, 0.81, 0.92, 0.84], dtype=torch.float32)

def iou_one(a, b):
    tl = torch.max(a[:2], b[:2]); br = torch.min(a[2:], b[2:])
    wh = (br - tl).clamp(min=0); inter = wh[0]*wh[1]
    areal=(a[2]-a[0])*(a[3]-a[1]); area2=(b[2]-b[0])*(b[3]-b[1])
    return inter/(areal+area2-inter+1e-7)

def nms(boxes, scores, th=0.5):
    keep=[]; idx = scores.argsort(descending=True)
    while idx.numel()>0:
        i = idx[0]; keep.append(i.item())
        if idx.numel()==1: break
        ov = torch.stack([iou_one(boxes[i], boxes[j]) for j in idx[1:]]
        idx = idx[1:][ov<=th]
    return keep

def soft_nms(boxes, scores, th=0.5, sigma=0.5, score_th=0.001):
    idx = scores.argsort(descending=True)
    b = boxes.clone(); s = scores.clone(); keep=[]
    while idx.numel()>0:
        i = idx[0]; keep.append(i.item())
        if idx.numel()==1: break
        others = idx[1:]
        ov = torch.stack([iou_one(b[i], b[j]) for j in others])
        s[others] = s[others]*torch.exp(-(ov**2)/sigma)
        idx = others[s[others]>score_th]
```

```

        idx = idx[s[idx].argsort(descending=True)]
    return keep

k1 = nms(boxes, scores, 0.5)
k2 = soft_nms(boxes, scores, 0.5, 0.5, 0.05)
print('NMS keep:', k1, '| Soft-NMS keep:', k2)

draw_boxes(IMG_PATH, boxes.numpy(), scores=scores.numpy(), out=root_dir
+'runs_demo/a2_raw.jpg')
draw_boxes(IMG_PATH, boxes.numpy()[k1], scores=scores.numpy()[k1], color=(0,255,0),
out=root_dir +'runs_demo/a2_nms.jpg')
draw_boxes(IMG_PATH, boxes.numpy()[k2], scores=scores.numpy()[k2], color=(0,0,255),
out=root_dir +'runs_demo/a2_softnms.jpg')

```

B.3 Anchors / Grid YOLO (trực quan hóa tâm lưới + anchor Boxes)

Mục tiêu: Trực quan hóa việc chia lưới và dựng Anchor Boxes theo các ô lưới trong Yolo

Kỳ vọng: thấy mạng lưới tâm ô và vài anchor quanh 1 tâm.

```

im = cv2.imread(IMG_PATH)
H, W, chal_ = im.shape

feat_h, feat_w, stride = 40, 60, 16 # ví dụ P3 stride 16
yv, xv = torch.meshgrid(torch.arange(feat_h), torch.arange(feat_w), indexing='ij')
grid = torch.stack((xv, yv), 2).view(-1,2).float()
centers = (grid + 0.5) * stride # (N,2) pixel

# Vẽ 200 tâm đầu tiên
im = cv2.imread(IMG_PATH)
for cx,cy in centers[:]:
    cv2.circle(im, (int(cx),int(cy)), 1, (0,0,255), -1)

# Vẽ 1-2 anchor rectangles quanh một tâm chọn
cx,cy = centers[800] # tùy ô
for w,h in [(40,40), (80,40), (40,80)]:
    x1,y1 = int(cx - w/2), int(cy - h/2)
    x2,y2 = int(cx + w/2), int(cy + h/2)
    cv2.rectangle(im, (x1,y1), (x2,y2), (0,255,255),1)

#os.makedirs('runs_demo', exist_ok=True)
cv2.imwrite(root_dir + 'runs_demo/a3_grid_anchors.jpg', im)

print('Saved: runs_demo/a3_grid_anchors.jpg')

```

B.4 FPN: trực quan “độ kích hoạt” (heatmap) trên các mức P3/P4/P5

Kỳ vọng: sẽ thấy heatmap P3 (đồ nhỏ/chi tiết), P4, P5 (ngữ nghĩa mạnh hơn, thô hơn) – Tương tự Lab04 nhưng sẽ áp heatmap của chúng lên ảnh gốc

```

. # ==== B.4 (FIXED) FPN: trực quan “độ kích hoạt” P3/P4/P5 ====

import cv2, numpy as np, torch, torch.nn as nn
import torch.nn.functional as F
from torchvision.models import mobilenet_v3_small, MobileNet_V3_Small_Weights

# 1) Đọc ảnh an toàn (tránh stride âm)

img_bgr = cv2.imread(IMG_PATH)
assert img_bgr is not None, f"Không đọc được ảnh: {IMG_PATH}"
img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# (Tuỳ chọn) pad ảnh để H, W chia hết cho 32 giúp downsample ổn định hơn
def pad_to_divisible(im, div=32):
    h, w = im.shape[:2]
    nh = (h + div - 1) // div * div
    nw = (w + div - 1) // div * div
    if (nh, nw) == (h, w):
        return im
    pad_b = nh - h
    pad_r = nw - w
    # Pad phía dưới/phải để không lệch nội dung
    return cv2.copyMakeBorder(im, 0, pad_b, 0, pad_r, borderType=cv2.BORDER_REPLICATE)

img = pad_to_divisible(img, 32)
t = torch.from_numpy(img).permute(2,0,1).float()/255.0
t = t.unsqueeze(0)

```

```
# 2) Backbone nhẹ để demo
base = mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT).features.eval()

# Lấy vài mốc làm "C3/C4/C5" (chỉ để minh họa; MobileNet không có C-stages chuẩn như ResNet)
# Bạn có thể đổi danh sách taps tùy phiên bản torchvision
taps = [3, 6, 12] # các chỉ số layer trong base.features
def extract_c3c4c5(x):
    feats = {}
    y = x
    for i, m in enumerate(base):
        y = m(y)
        if i in taps:
            feats[i] = y
    # đảm bảo đủ 3 mức
    assert all(k in feats for k in taps), f"Không lấy đủ taps {taps}, có {list(feats.keys())}"
    return feats[taps[0]], feats[taps[1]], feats[taps[2]]

with torch.no_grad():
    C3, C4, C5 = extract_c3c4c5(t)

class SimpleFPN(nn.Module):
    def __init__(self, c3, c4, c5, out=128):
        super().__init__()
        self.l3 = nn.Conv2d(c3, out, 1)
        self.l4 = nn.Conv2d(c4, out, 1)
        self.l5 = nn.Conv2d(c5, out, 1)
        self.s3 = nn.Conv2d(out, out, 3, padding=1) # <-- dùng padding, không phải p=
        self.s4 = nn.Conv2d(out, out, 3, padding=1)

    def forward(self, C3, C4, C5):
        P5 = self.l5(C5)
        # UpSample theo kích thước chuẩn của nhánh lateral để tránh lệch 1 pixel
        P4 = self.l4(C4) + F.interpolate(P5, size=C4.shape[-2:], mode='nearest')
        P3 = self.l3(C3) + F.interpolate(P4, size=C3.shape[-2:], mode='nearest')
        P4 = self.s4(P4)
        P3 = self.s3(P3)
        return P3, P4, P5

with torch.no_grad():
    fpn = SimpleFPN(C3.shape[1], C4.shape[1], C5.shape[1], out=64).eval()
    P3, P4, P5 = fpn(C3, C4, C5)

# 3) Vẽ heatmap
def heat_to_img(h, base_img_bgr, out_path):
    # h: [1,C,H,W] -> trung bình kênh -> normalize -> resize -> overlay
    hm = h.abs().mean(1, keepdim=True) # [1,1,H,W]
    hm = (hm - hm.min()) / (hm.max() - hm.min() + 1e-7)
    hm = hm[0,0].cpu().numpy()
    H, W = base_img_bgr.shape[:2]
    hm = cv2.resize(hm, (W, H))
    heat = (hm*255).astype(np.uint8)
    heat = cv2.applyColorMap(heat, cv2.COLORMAP_JET)
    blend = cv2.addWeighted(base_img_bgr, 0.6, heat, 0.4, 0)
    cv2.imwrite(out_path, blend)
    print("Saved:", out_path)

# Chú ý: base_img_bgr cần trùng kích thước với ảnh đã pad
base_img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
import os
#os.makedirs('runs_demo', exist_ok=True)
heat_to_img(P3, base_img_bgr, root_dir + 'runs_demo/a4_p3.jpg')
heat_to_img(P4, base_img_bgr, root_dir + 'runs_demo/a4_p4.jpg')

heat_to_img(P5, base_img_bgr, root_dir + 'runs_demo/a4_p5.jpg')
```

B.5 “Head” SSD: kiểm tra hình dạng đầu ra (cls/reg) & sanity box-regression

Kỳ vọng: khi offsets = 0, hộp decode ~ đúng anchor (bài test sanity head/reg).

Bài tập: thêm noise nhỏ vào reg_sel (ví dụ + 0.05*torch.randn_like(reg_sel)) để thấy hộp lệch đi.

```
import torch.nn as nn

class SSDHeadDemo(nn.Module):
```

```

def __init__(self, in_ch, num_anchors, num_classes):
    super().__init__()
    self.cls = nn.Conv2d(in_ch, num_anchors*num_classes, 3, padding=1)
    self.reg = nn.Conv2d(in_ch, num_anchors*4, 3, padding=1)
def forward(self, f):
    B,C,H,W = f.shape
    cls = self.cls(f).permute(0,2,3,1).contiguous().view(B, -1, num_classes)
    reg = self.reg(f).permute(0,2,3,1).contiguous().view(B, -1, 4)
    return cls, reg

num_classes = 4+1 # background + 4 lớp ví dụ
num_anchors = 6 # ví dụ 6 anchor/mỗi ô
fake_feat = torch.randn(1, 64, 40, 60) # giống P3 ở trên
head = SSDHeadDemo(64, num_anchors, num_classes)
cls_out, reg_out = head(fake_feat)
print('cls_out:', cls_out.shape, 'reg_out:', reg_out.shape) # -> (1, H*W*num_anchors,
num_classes) & (1, *, 4)

# Sanity: thử reg_out + anchors -> decode ra boxes, rồi vẽ thử
# Tạo anchors tương ứng với (H,W) và stride ~16 như A.3
#H,W = 640,960; feat_h,feat_w,stride = 40,60,16

im = cv2.imread(IMG_PATH)
H, W, chal_ = im.shape
feat_h,feat_w,stride = 40,60,16

yv,xv = torch.meshgrid(torch.arange(feat_h), torch.arange(feat_w), indexing='ij')
centers = (torch.stack((xv,yv),2).view(-1,2).float()+0.5)*stride
anchors=[]
for (w,h) in [(40,40), (80,40), (40,80), (64,64), (96,48), (48,96)]:
    cxcywh = torch.cat([centers, torch.full_like(centers, w), torch.full_like(centers, h)],
dim=1)
    # nhưng cần (cx,cy,w,h) - chuyển sang xxyy:
    for i in range(cxcywh.shape[0]):
        cx,cy= cxcywh[i,0], cxcywh[i,1]
        anchors.append([cx-0.5*w, cy-0.5*h, cx+0.5*w, cy+0.5*h])
anchors = torch.tensor(anchors, dtype=torch.float32)

# Lấy N điểm reg bất kỳ & giả lập decode (dùng ssd_decode ở A.1)
def ssd_decode_xyxy(t, anc_xyxy, v=(0.1,0.2)):
    v1,v2=v
    def xxyy2xywh(b):
        x1,y1,x2,y2 = b.unbind(-1); w=x2-x1; h=y2-y1
        cx=x1+0.5*w; cy=y1+0.5*h
        return torch.stack([cx,cy,w,h],-1)
    def xywh2xxyy(b):
        cx,cy,w,h = b.unbind(-1)
        x1=cx-0.5*w; y1=cy-0.5*h; x2=cx+0.5*w; y2=cy+0.5*h
        return torch.stack([x1,y1,x2,y2],-1)
    anc = xxyy2xywh(anc_xyxy)
    cxcy = t[:,2]*v1*anc[:,2:] + anc[:,2:]
    wh = torch.exp(t[:,2]*v2)*anc[:,2:]
    return xywh2xxyy(torch.cat([cxcy,wh],-1))

sel = torch.randint(0, reg_out.shape[1], (200,)) # 200 anchor ngẫu nhiên
reg_sel = reg_out[0, sel] * 0.0 # đặt offset=0 để decode ~ về chính anchor (sanity)
boxes_dec = ssd_decode_xyxy(reg_sel, anchors[sel])
#draw_boxes(IMG_PATH, boxes_dec.numpy(), color=(255,0,0), out=root_dir
+'runs_demo/a5_head_sanity.jpg')
draw_boxes(IMG_PATH, boxes_dec.detach().numpy(), color=(255,0,0), out=root_dir
+'runs_demo/a5_head_sanity.jpg')

```

Phần code trên đây đang thực hiện với các boundingbox ground truth (Bbox GT) được lấy theo một số đối tượng có trong ảnh street.jpg gửi kèm. Nếu dùng file ảnh khác các bạn nên chọn Bbox GT phù hợp cho các cell B0 và B2.

Bài tập mở rộng:

Dùng pretrained YOLOv7 (hoặc v8) chạy inference -> lấy **boxes, scores** thật trên ảnh rồi áp **Soft-NMS** (B.2) để thấy khác biệt rõ hơn.

- Thử **tile-based detection**: chia ảnh lớn thành ô 1024×1024, ghép kết quả và áp NMS toàn cục.
- Với **FPN**: thay backbone khác (ResNet50) và đổi “tap points” để xem heatmap P3/P4/P5 khác nhau thế nào.

4. CHẠY MÔ HÌNH PRETRAINED NHANH (INFERENCE)

4.1 CHẠY YOLOv8 VỚI PyTorch, GÓI ultralytics

Mục tiêu: Phần này chỉ chạy dự đoán với weight có sẵn của YOLOv8 từ **PyTorch, gói ultralytics**. Các bạn có thể thực nghiệm với các bản khác của YOLO (ví dụ v3, 5, 7, X...)

Các mô hình họ **YOLO** khá hiệu quả nên nhiều thư viện cung cấp code của mô hình này. Trong file đính kèm yolo3_cv2_model1.ipynb là ví dụ về mô hình YOLOv3 với thư viện Python-OpenCV. Weights pretrained đã được cung cấp trong các tệp đính kèm, các bạn tải xuống để thực nghiệm.

```
model = YOLO('yolov8n.pt') # n/s/m/l/x
# Inference 1 ảnh

img_path = "D:/Teach_n_Train/Advanced_Lessons_CV/LABS/r-cnn/faster-rcnn/traffic.jpg"
filename = os.path.basename(img_path)

res = model(img_path) # hoặc link/np.ndarray
res[0].show() # hiển thị với boxes/labels

# Lấy kết quả dạng tensors:
boxes = res[0].boxes.xyxy
scores = res[0].boxes.conf
cls = res[0].boxes.cls
```

Bài tập: Sử dụng chương trình này để tạo ra Bbox GT cho cell B0 ở phần 3, sau đó thực hiện lại để kiểm tra các thuật toán Bbox regression và NMS của phần đó, lưu lại kết quả.

4.2 CHẠY SSD VỚI PyTorch, WEIGHTs PRETRAINED BACKBONE SSD300_VGG16_Weights

Lưu ý trong thư viện TORCH, mô hình SSD chỉ được cung cấp 02 bộ trọng số pretrained với Backbone là VGG16 và SSDLite320_MobileNet_V3_Large_Weights. Nếu muốn đổi Backbone các bạn phải tự dựng mô hình và train lại. Dưới đây là ví dụ sử dụng Backbone VGG16.

```
OBJECT_NAMES = ['_background_', 'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A', 'stop sign', 'parking meter',
'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
'N/A', 'backpack', 'umbrella', 'N/A', 'N/A', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis',
'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove', 'skateboard',
'surfboard', 'tennis racket', 'bottle', 'N/A', 'wine glass', 'cup', 'fork', 'knife', 'spoon',
'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza',
'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed', 'N/A', 'dining table', 'N/A', 'N/A',
'toilet', 'N/A', 'tv', 'laptop', 'mouse', 'remote', 'keyboard', 'cellone', 'microwave', 'oven',
'toaster', 'sink', 'refrigerator', 'N/A', 'book', 'clock', 'vase', 'scissors', 'teddy bear',
'hair drier', 'toothbrush']

import torch, torchvision
from torchvision.models.detection import ssd300_vgg16, SSD300_VGG16_Weights
from PIL import Image, ImageDraw, ImageFont
import torchvision.transforms as T

import numpy as np
import os
import matplotlib.pyplot as plt
```

```

%matplotlib inline

# Load a font (replace with your font file path)
try:
    font = ImageFont.truetype("arial.ttf", 15)
except IOError:
    font = ImageFont.load_default()

weights = SSD300_VGG16_Weights.DEFAULT
model = ssd300_vgg16(weights=weights).eval()
pre = weights.transforms()

img_path = "D:/Teach_n_Train/Advanced_Lessons_CV/LABS/r-cnn/faster-rcnn/traffic.jpg"
filename = os.path.basename(img_path)

img = Image.open(img_path).convert('RGB')
x = pre(img).unsqueeze(0)
with torch.no_grad():
    out = model(x)[0] # dict: 'boxes', 'scores', 'labels'

print(out['boxes'][:5], out['scores'][:5], out['labels'][:5])

boxes = out["boxes"].cpu().numpy()
labels = out["labels"].cpu().numpy()
scores = out["scores"].cpu().numpy()

draw = img.copy()
d = ImageDraw.Draw(draw)
for b,l,s in zip(boxes, labels, scores):
    if s < 0.5: continue
    x1,y1,x2,y2 = b
    d.rectangle([x1,y1,x2,y2], outline=(255,0,0), width=3)
    d.text((x1,y1), f"{OBJECT_NAMES[int(l)]}:{s:.2f}", font = font, fill=(255,255,0))

output_path = "D:/Teach_n_Train/Advanced_Lessons_CV/LABS/Object_Detection/Lab04/outputs"
os.makedirs(output_path, exist_ok=True)

draw.save(output_path + "/torch_infer_SSD_" + filename)

plt.imshow(np.asarray(draw))
plt.title("Image with bounding boxes")
plt.axis('off')
plt.show()

```

Các bạn hãy tự thay đổi backbone bằng các lệnh

```

from torchvision.models.detection import ssdlite320_mobilenet_v3_large,
                                     SSDLite320_MobileNet_V3_Large_Weights
weights = SSDLite320_MobileNet_V3_Large_Weights.DEFAULT
model = ssdlite320_mobilenet_v3_large(weights=weights)

```

sau đó điều chỉnh các tham số ngưỡng và so sánh kết quả.

5. FINE-TUNE NHANH (TRÊN PYTORCH)

5.1 FINE-TUNE YOLOV8 TRÊN DỮ LIỆU YOLO FORMAT

Trước hết để thống nhất, chúng ta tổ chức lại dữ liệu theo cây thư mục sau

```

dataset/
├─ images/train/*.jpg
├─ images/val/*.jpg
├─ labels/train/*.txt # mỗi dòng: class cx cy w h (normalized)
└─ labels/val/*.txt

```

Sau đó ta ghi cấu trúc đó vào file yaml

```
data.yaml:

train: dataset/images/train
val: dataset/images/val
nc: <num_classes>
names: [class0, class1, ...]
```

Huấn luyện & đánh giá: Chúng ta sẽ sử dụng luôn gói **ultralytics** và huấn luyện lại mô hình YOLOv8. Giả sử cấu trúc các file huấn luyện (gồm ảnh và annotation tương ứng) đã được lưu như trong file Yaml nói trên. Code sử dụng weights pretrained của YOLOv8n, đặt số epochs là 100, image size đầu vào là width = 640. Các bạn có thể đặt lại để phù hợp với cấu hình thực nghiệm, cũng như bổ sung các đoạn code để lưu weights, hiển thị kết quả train, chạy inference (đã có ở phần trước).

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt') # load backbone+head pretrained COCO
model.train(data='data.yaml',
            imgsz=640, epochs=100, batch=16,
            lr0=0.01, cosine=True, mosaic=1.0, hsv=True,
            device=0)
model.val()
model.export(format='torchscript') # tùy chọn: onnx, tflite, openvino...
```

Bài tập: bật/tắt mosaic (đặt = 0), thay image size là 512/768, thử với các mô hình yolov8s/m (thay weights) và so sánh mAP50-95.

5.2 FINE-TUNE SSD300 (TORCHVISION) TRÊN DỮ LIỆU VOC-STYLE

Để thực hiện huấn luyện lại/tinh chỉnh cho SSD (thư viện PyTorch), giả sử dữ liệu vào là dạng VOC, lúc đó ta cần chuyển sang dữ liệu cho TORCH. Đoạn code dưới đây trước hết chuẩn bị dataset VOC (JPEGImages/Annotations) hoặc/và chuyển nhãn sang torchvision format (ở đây tôi chỉ cung cấp như một *Dataloader* tối giản VOCDataset, các bạn cần bổ sung thêm các phần khác nếu cần thiết). Sau đó ta xây dựng mô hình SSD với backbone là VGG16 dựa trên weights đã có, rồi bắt đầu tinh chỉnh hoặc huấn luyện lại toàn bộ.

```
import os, torch, torchvision
from torchvision.models.detection import ssd300_vgg16, SSD300_VGG16_Weights
from torchvision.transforms import functional as F
import xml.etree.ElementTree as ET
from PIL import Image

CLASSES = ["__background__", "person", "car", "bus", "bike"] # ví dụ

def parse_voc_xml(xml_path):
    root = ET.parse(xml_path).getroot()
    boxes, labels = [], []
    for obj in root.findall('object'):
        name = obj.find('name').text
        bnd = obj.find('bndbox')
        x1, y1, x2, y2 = [float(bnd.find(t).text) for t in ['xmin', 'ymin', 'xmax', 'ymax']]
        boxes.append([x1, y1, x2, y2])
        labels.append(CLASSES.index(name))
    return torch.tensor(boxes, dtype=torch.float32), torch.tensor(labels, dtype=torch.int64)

class VOCDataset(torch.utils.data.Dataset):
    def __init__(self, img_dir, ann_dir):
        self.imgs = sorted([f for f in os.listdir(img_dir) if f.endswith('.jpg')])
        self.img_dir, self.ann_dir = img_dir, ann_dir
    def __len__(self): return len(self.imgs)
    def __getitem__(self, i):
        img_path = os.path.join(self.img_dir, self.imgs[i])
        ann_path = os.path.join(self.ann_dir, self.imgs[i].replace('.jpg', '.xml'))
        img = Image.open(img_path).convert('RGB')
        boxes, labels = parse_voc_xml(ann_path)
        target = {'boxes': boxes, 'labels': labels}
        img = F.to_tensor(img) # SSD weights đã định nghĩa transforms riêng khi dùng weights
        return img, target
```

```
def collate_fn(batch): return tuple(zip(*batch))

train_set = VOCDataset('VOC/train/JPEGImages', 'VOC/train/Annotations')
val_set = VOCDataset('VOC/val/JPEGImages', 'VOC/val/Annotations')
train_loader = torch.utils.data.DataLoader(train_set, batch_size=8, shuffle=True,
collate_fn=collate_fn)
val_loader = torch.utils.data.DataLoader(val_set, batch_size=8, shuffle=False,
collate_fn=collate_fn)

weights = SSD300_VGG16_Weights.DEFAULT
model = ssd300_vgg16(weights=weights)
# Sửa num_classes cho phù hợp (background + N lớp)
num_classes = len(CLASSES)
model.head.classification_head.num_classes = num_classes
# Khởi tạo lại lớp cuối nếu cần:
model.head.classification_head.cls_logits = torch.nn.Conv2d(1024,
model.head.classification_head.num_anchors * num_classes, 3, padding=1)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
opt = torch.optim.SGD(model.parameters(), lr=1e-3, momentum=0.9, weight_decay=5e-4)
lr_sch = torch.optim.lr_scheduler.StepLR(opt, step_size=8, gamma=0.1)

for epoch in range(15):
    model.train()
    for imgs, targets in train_loader:
        imgs = [i.to(device) for i in imgs]
        targets = [{k:v.to(device) for k,v in t.items()} for t in targets]
        loss_dict = model(imgs, targets)
        loss = sum(loss_dict.values())
        opt.zero_grad(); loss.backward(); opt.step()
    lr_sch.step()
    model.eval()
    # TODO: tính mAP (có thể dùng pycocotools nếu bạn chuyển định dạng); tạm thời in loss
    print(f"Epoch {epoch+1}: train_loss={loss.item():.3f}")

torch.save(model.state_dict(), 'ssd300_finetuned.pth')
```

Bài tập: đóng băng backbone vài stage, thử các scheduler (CosineAnnealingLR/ReduceLROnPlateau), thêm **early stopping** (xem bài phần Faster-RCNN) theo loss/val-mAP.

5.3 FINE-TUNE SSD300 (TORCHVISION) VỚI BACKBONE MỚI

Trong thư viện TORCH, mô hình SSD chỉ được cung cấp pretrained weights với 02 backbone là SSD300_VGG16 hoặc **SSDLite320_MobileNetV3_Large**. Backbone thứ hai nhẹ hơn cũng như nhạy cảm hơn với các vật thể nhỏ. Tập đính kèm lab_ssd_one_stage_plus.ipynb là code mô hình SSD trên torch cho phép chọn lựa weights (từ 02 bộ nói trên).

Thay đổi chính so với code phần trên:

- Cho phép **chọn backbone**: SSD300_VGG16 hoặc **SSDLite320_MobileNetV3_Large** (nhẹ hơn, thường mạnh hơn VGG cho vật thể nhỏ).
- Thêm **fine-tune** có **Early Stopping** + **LR Scheduler** (CosineAnnealingLR hoặc StepLR).
- Thêm phần **đánh giá mAP (COCO)** dùng pycocotools, kèm **converter VOC→COCO JSON** tối giản để bạn có thể evaluate ngay khi nhận đang ở dạng VOC XML.

Cách dùng nhanh:

- Ở phần “Choose Backbone”, đổi BACKBONE = 'ssdlite_mnv3' hoặc 'vgg16'.
- Nếu fine-tune: chỉnh đường dẫn VOC/train và VOC/val (JPEGImages/Annotations) + danh sách CLASSES.
- mAP: nếu nhận là VOC, cell “VOC→COCO” sẽ tự tạo coco_gt_val.json, chạy inference để sinh coco_dt_val.json, rồi tính mAP bằng pycocotools.

Trong trường hợp không sử dụng 02 backbone trên, chúng ta bắt buộc phải tự xây dựng mô hình bằng cách ghép phần Backbone chúng ta muốn và SSD Head.

Điều này là do nếu ta tự build SSD và dù chỉ “thay backbone” (ResNet/EfficientNet/Xception...) rồi **load pretrain ImageNet** cho backbone, thì **các phần còn lại của SSD (extra feature layers, head cls/reg, default boxes...)** đều **chưa có trọng số phù hợp** → chúng ta **ít nhất phải huấn luyện (fine-tune) phần head**; thực tế để có mAP tốt, thường cần **fine-tune cả mạng** (đóng băng backbone vài epoch đầu rồi mở dần). Tuy nhiên để chọn giải pháp “phải train lại ở mức ít hơn”, bằng cách chỉ huấn luyện lại cho phần HEAD của SSD.

Dưới đây là một số điểm chú ý:

Vì sao phải train lại?

- **Head phát hiện (cls/reg)** là mới tinh (random init) → không thể suy luận hộp/bậc xác suất ngay.
- **Extra layers/FPNlite** tạo pyramid (P3...P7) cũng mới tinh nếu bạn tự thêm.
- **Priors/Default boxes** (tỷ lệ, tỉ lệ khung) thay đổi → phân phối logit/loss khác, cần học lại.
- Chuyển từ **pretrain classification** → **detection** là **transfer task** khác mục tiêu (global label → dense boxes).

Các lựa chọn thực tế (từ “ít train” → “train nhiều”)

1. **Freeze backbone** (pretrained ImageNet) + **train head + extra layers** (10–30 epoch)
 - Nhanh, mAP tăng chậm; phù hợp khi dữ liệu ít.
2. **Warmup**: freeze backbone 5–10 epoch → **unfreeze 1–3 stage cuối** (30–100 epoch)
 - Thường là **điểm cân bằng tốt** giữa thời gian và chất lượng.
3. **Fine-tune full** (unfreeze toàn bộ, LR nhỏ hơn cho backbone)
 - Tốt nhất khi dataset đủ lớn ($\geq 10k$ ảnh).
4. **Khởi tạo từ detector đã pretrained** (VD: RetinaNet-ResNet50-FPN) rồi gắn head SSD tương thích
 - Vẫn phải fine-tune, nhưng **hội tụ nhanh hơn** so với chỉ ImageNet.

Công thức khởi tạo & train khuyến nghị

- **LR khác nhau**: $lr_backbone = 0.1 * lr_head$ (hoặc dùng optimizer param groups).
- **Scheduler**: CosineAnnealingLR hoặc StepLR; có **warm-up 500–1000 iteration** đầu.
- **Early stopping** trên val mAP hoặc val loss.
- **Hard Negative Mining** (SSD) hoặc Focal Loss (nếu bạn tùy biến).

Khi nào có thể “ít train”?

- Nếu chúng ta **giữ nguyên** toàn bộ **kiến trúc SSD** (bao gồm vị trí/chiều kênh các feature levels) và **chỉ thay “thân” nhưng feature shapes tương thích** với một mô hình đã pretrained SSD *cùng cấu trúc*, khi đó ta có thể “transplant” một phần trọng số head — nhưng trên thực tế, đổi backbone gần như luôn làm **đổi phân phối & shape feature**, khiến việc cấy nguyên vẹn head gần như **không khả thi**. Kết luận: **vẫn phải fine-tune**.

Phác thảo code (torchvision) cho SSD + ResNet50 (chỉ là khung tối thiểu)

Dưới đây là phác thảo một khung các bước cần thiết để thay Backbone ResNet50 vào mô hình SSD (kết hợp đầu ra Resnet vào đầu vào của SSD Head). Chú ý phần SSD Head ở đây chỉ được cung cấp ở mức sơ lược.

```
import torch
import torch.nn as nn
from collections import OrderedDict
from torchvision.models import resnet50, ResNet50_Weights
from torchvision.models.detection.anchor_utils import DefaultBoxGenerator
from torchvision.models.detection.ssd import SSD, SSDHead
from torchvision.ops import MultiScaleRoIAlign # (không dùng cho SSD; chỉ để minh họa import)
from torchvision.ops.misc import Conv2dNormActivation

# 1) Backbone: lấy feature maps tại các stride mong muốn (C3,C4,C5)
class ResNetBackbone(nn.Module):
    def __init__(self, out_channels=256):
        super().__init__()
        base = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
        # tách các stage
        self.body = nn.Sequential(
            base.conv1, base.bn1, base.relu, base.maxpool,
            base.layer1, # C2 stride 4
```

```

        base.layer2, # C3 stride 8
        base.layer3, # C4 stride 16
        base.layer4 # C5 stride 32
    )
    # giảm kênh về cùng chiều (tùy chọn)
    self.lateral3 = nn.Conv2d(512, out_channels, 1) # C3
    self.lateral4 = nn.Conv2d(1024, out_channels, 1) # C4
    self.lateral5 = nn.Conv2d(2048, out_channels, 1) # C5

    # extra layers để có thêm pyramid (P6,P7) kiểu SSD
    self.extra6 = Conv2dNormActivation(out_channels, out_channels, kernel_size=3, stride=2,
padding=1)
    self.extra7 = Conv2dNormActivation(out_channels, out_channels, kernel_size=3, stride=2,
padding=1)

    def forward(self, x):
        # chạy đến các mốc ResNet
        for i, m in enumerate(self.body):
            x = m(x)
            if i == 6: # sau layer2 -> C3
                C3 = x
            elif i == 7: # sau layer3 -> C4
                C4 = x
            # i==8 sau layer4 -> C5 (nhưng body kết thúc ở layer4)
        C5 = x
        P3 = self.lateral3(C3)
        P4 = self.lateral4(C4)
        P5 = self.lateral5(C5)
        P6 = self.extra6(P5)
        P7 = self.extra7(P6)
        return OrderedDict([
            ("0", P3), # stride ~8
            ("1", P4), # ~16
            ("2", P5), # ~32
            ("3", P6), # ~64
            ("4", P7), # ~128
        ])

# 2) Default boxes (kích thước & tỉ lệ cho từng mức)
box_generator = DefaultBoxGenerator(
    [[2, 3, 1/2], [2, 3, 1/2], [2, 3, 1/2], [2, 3, 1/2], [2, 3, 1/2]] # aspect ratios per
level
)
# sizes theo pixel hoặc theo tỉ lệ ảnh (tùy phiên bản torchvision); có thể để None & cấu hình
qua model

# 3) SSD model: backbone trả OrderedDict[str, Tensor]
num_classes = 1 + 4 # background + N lớp
backbone = ResNetBackbone(out_channels=256)
# mỗi feature map -> số anchor/ô; SSDHead sẽ suy ra từ box_generator
model = SSD(backbone, num_classes=num_classes, anchor_generator=box_generator, size=(640, 640))
# Khởi tạo head (nếu cần tùy biến nhiều hơn)
model.head = SSDHead(backbone_out_channels=[256,256,256,256,256],
                    num_anchors=box_generator.num_anchors_per_location(),
                    num_classes=num_classes)

# 4) Khởi tạo trọng số head/extra (Kaiming) & load ImageNet cho backbone đã làm trong ResNet
def init_head(m):
    if isinstance(m, nn.Conv2d):
        nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
        if m.bias is not None: nn.init.constant_(m.bias, 0)
model.head.apply(init_head)

```

Tóm lại:

- Chỉ thay backbone + load ImageNet **không đủ** để suy luận: **bắt buộc train lại head** (và thường cả extra/FPN).
- Lộ trình khuyên dùng: **freeze backbone 5–10 epoch** → **mở dần stage cuối** → **fine-tune** toàn mạng với LR nhỏ hơn cho backbone, có **Cosine LR + warmup + early stopping**.
- Điều chỉnh **priors (sizes/ratios)** theo dataset của bạn (nếu bạn tự xây dựng dữ liệu train) để tăng mAP đáng kể.

Bài tập: Hãy thực hiện thay Backbone tương tự cho một mô hình YOLO nào đó chưa có weights tương ứng trong TORCH, chọn các Backbone mới và hiệu quả.

6. FINE-TUNE CÁC MÔ HÌNH TRÊN TENSORFLOW.KERAS

Hiện tại các thư viện TensorFlow có một số phần không tương thích cho các mô hình chúng ta vừa đề cập. Trong trường hợp cần thực thi trên thư viện này, chúng ta có thể sử dụng API của gói Object Detection (OD API) để xây dựng. Chúng ta cần kiểm tra xem requirements của các mô hình có tương thích với phiên bản của TensorFlow trên máy hay không.

6.1 SSD MOBILENET V2 VỚI TENSORFLOW OBJECT DETECTION API (TF2)

Bước tóm tắt:

- Cài đặt TF OD API; 2) Chuẩn bị TFRecord; 3) label_map.pbtxt;
- Tải checkpoint SSD (MobileNetV2); 5) Tạo pipeline.config;
- Train bằng model_main_tf2.py; 7) Export exporter_main_v2.py; 8) Inference.

Cài đặt & import (Colab/local TF2):

```
# (gợi ý) trên Colab: !pip install -q tf-models-official==2.* tensorflow_io
# Hoặc clone tensorflow/models và cài đặt object_detection theo hướng dẫn.
```

Chuyển dữ liệu → TFRecord (ví dụ PennFudanPed/COCO-tùy bạn):

- Viết script tạo TFExample: ảnh (encoded), bboxes (xmin/xmax/ymin/ymax normalized), class text/id.
- Tạo train.record và val.record.

Tập label_map.pbtxt mẫu:

```
item { id: 1 name: "person" }
item { id: 2 name: "car" }
```

Train:

```
# ví dụ:
python model_main_tf2.py \
  --model_dir=checkpoints/ssd_mnv2 \
  --pipeline_config_path=configs/ssd_mobilenet_v2_fpn-lite_320x320_coco17_tpu-8.config
```

Export:

```
python exporter_main_v2.py \
  --input_type image_tensor \
  --pipeline_config_path configs/ssd_mobilenet_v2_fpn-lite_320x320_coco17_tpu-8.config \
  --trained_checkpoint_dir checkpoints/ssd_mnv2 \
  --output_directory export/ssd_mnv2_savedmodel
```

Inference (SavedModel):

```
import tensorflow as tf, cv2
detect_fn = tf.saved_model.load('export/ssd_mnv2_savedmodel/saved_model')
img = cv2.imread('samples/street.jpg')[:, :, :-1]
input_tensor = tf.convert_to_tensor(img)[tf.newaxis, ...]
outs = detect_fn(input_tensor)
print([k for k in outs.keys()])
```

Bài tập: thử 320×320 vs 640×640, thay data augment, bật/đổi matcher (IOU threshold), thay FocalLoss.

7. BÀI TẬP MINH HOẠ TRÊN ẢNH “MAPS/STREET”

- Lấy 1 ảnh vệ tinh khu phố hoặc ảnh chụp giao thông của bạn (đặt ở thư mục như đã cấu trúc trong chương trình của bạn).

2. Chạy YOLOv8 pretrained, vẽ lại kết quả với NMS thường vs Soft-NMS (từ A.2), so sánh số lượng/điểm.
3. Thử tăng imgsz 640→960, quan sát thay đổi phát hiện xe nhỏ xa.
4. **Nâng cao:** crop theo tile 1024×1024 rồi hợp nhất kết quả (mẹo thường dùng với ảnh bản đồ lớn).

Module vẽ kết quả (PyTorch + OpenCV):

```
import cv2, numpy as np
from ultralytics import YOLO

def draw(img, boxes, labels=None, scores=None, color=(0,255,0)):
    im = img.copy()
    for i, b in enumerate(boxes):
        x1,y1,x2,y2 = map(int, b)
        cv2.rectangle(im, (x1,y1), (x2,y2), color, 2)
        if labels is not None:
            txt = str(labels[i])
            if scores is not None: txt += f" {scores[i]:.2f}"
            cv2.putText(im, txt, (x1, max(0,y1-5)), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
color, 2)
    return im

img = cv2.imread('samples/map.jpg')
model = YOLO('yolov8s.pt')
res = model(img[:,:,:-1], imgsz=960)[0]
b = res.boxes.xyxy.cpu().numpy()
s = res.boxes.conf.cpu().numpy()
c = res.boxes.cls.cpu().numpy().astype(int)
out = draw(img, b, labels=c, scores=s)
cv2.imwrite('samples/map_yolo_result.jpg', out)
```

8. CÂU HỎI ÔN TẬP

- So sánh **anchor-based** (SSD/YOLOv3/5) vs **anchor-free** (YOLOX/YOLOv8/FCOS): khác nhau ở đâu hồi quy (trực tiếp vs theo anchor).
- Tại sao 1-stage không cần RoI Align? (Không có proposal stage, dự đoán trực tiếp trên lưới/feature map).
- Vai trò **FPN** trong phát hiện vật nhỏ?
- Khi nào dùng **Soft-NMS**? Khi nào dùng **class-agnostic box head**?
- Lựa chọn **loss** cho bbox: L1/Smooth-L1 vs IoU/GIoU/DIoU/CIoU — ưu/nhược?